

University M'hamed Bougara-Boumerdes

Faculty of Sciences

IT departement



Algorithm 1

Course notes

Directed By :

Benabderrezak Youcef

-Phd student in Cyber security and future Pr-

y.brnabderrezak@univ-boumerdes.dz

Telegram : <https://t.me/infoumbb1>



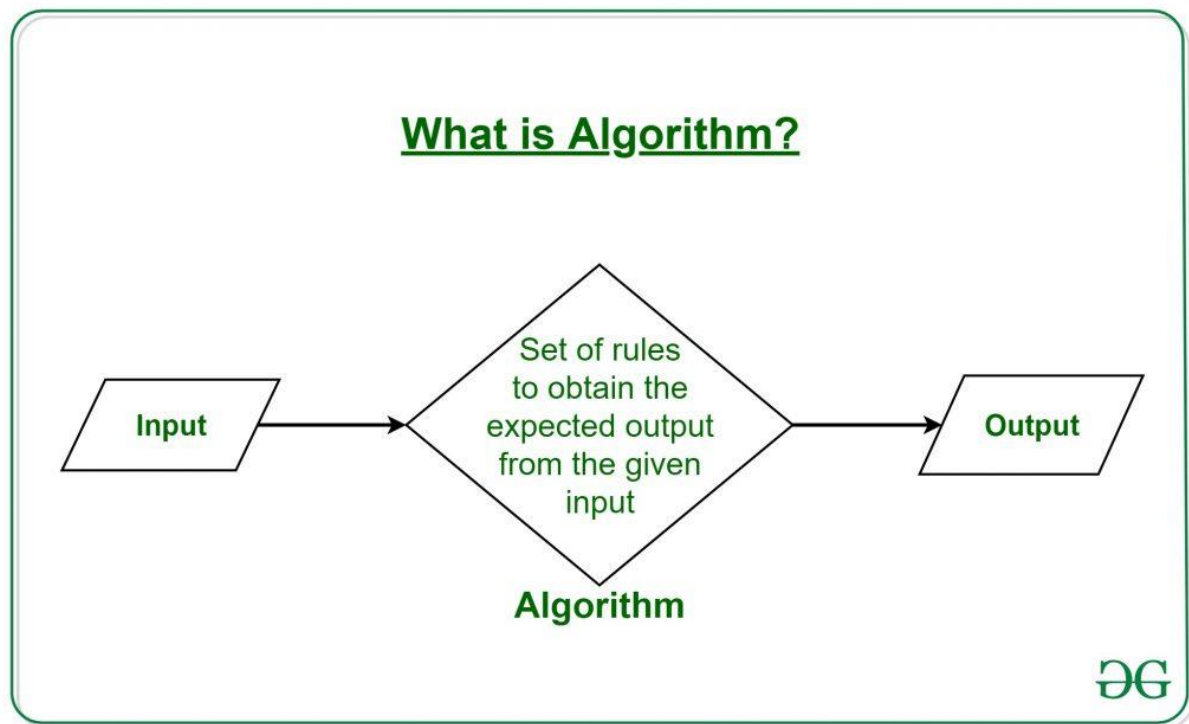
Table of contents

1. What is an Algorithm ?.....	3
2. Characteristics of an algorithm.....	3
3. The basics of an Algorithm.....	5
3.1. Basic structure	5
3.2. variables and constants.....	5
3.3. Integers	5
3.4. Floats / double	6
3.5. Characters.....	6
3.6. chain of characters (String).....	6
3.7. Comments.....	7
4. Conditions	8
4.1. If statement.....	8
4.2. If-else statement	8
4.3. Switch statement	9
5. Loops.....	11
5.1. For loop	11
5.2. While loop	12
5.3. Do-while loop.....	13
6. Arrays	14

6.2. One-dimensional (1-D) arrays	15
6.3. Two-dimensional (2-D) arrays.....	17
7. Functions	22
7.1. workflow of program when call afunction	24
7.2. Example of functions	25
7.3. Exercices	27

1. What is an Algorithm ?

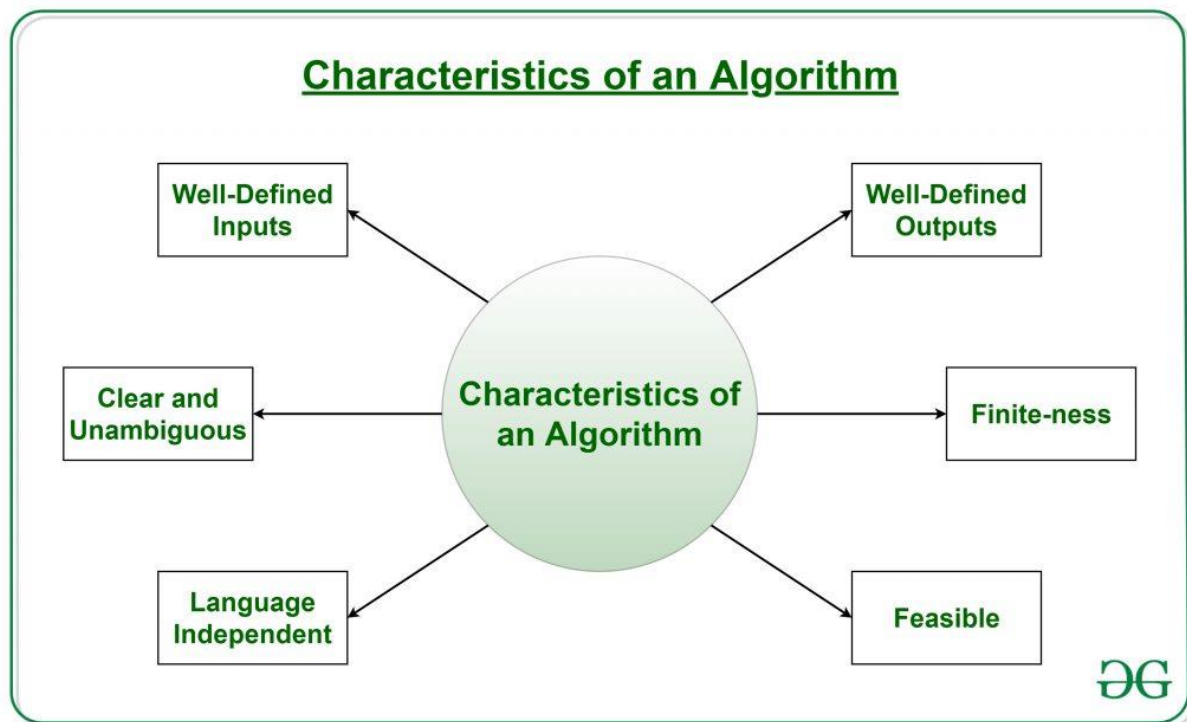
- ⇒ An algorithm is a **set of instructions** or a step-by-step procedure for **solving a problem** or **performing a specific task**.
- ⇒ It is a well-defined, finite sequence of computational steps that **takes some input** and **produces an output**



Algorithms are used in a wide range of applications, including computer programming, mathematics, engineering, and data analysis. They are essential for solving complex problems, processing large amounts of data, and automating tasks.

2. Characteristics of an algorithm

In order for some instructions to be an algorithm, it must have the following characteristics:



1. **Well-Defined Inputs:** If an algorithm asks for inputs, it should clearly state what inputs it needs, and they should make sense.
2. **Well-Defined Outputs:** The algorithm must clearly say what output it will produce, and it should be a sensible and well-defined result.
3. **Finite-ness:** The algorithm should have a clear endpoint, and it should not go on forever.
4. **Feasible:** The algorithm should be practical and able to be executed with the resources available.
5. **Language Independent:** The algorithm should be able to be understood and implemented in any programming language.
6. **Clear and unambiguous :** refer to instructions or descriptions in an algorithm that are easy to understand and do not have any ambiguity or confusion in their meaning.

3. The basics of an Algorithm

3.1. Basic structure

Algorithm Name;

2. **declaration of variables and constants**

3. **Statement of Duties**

4. **Start**

5.

6. **List of instructions**

7.

8. **End.**

3.2. variables and constants

- ⇒ In programming, variables and constants are used to store and manipulate data.
- ⇒ The main difference between them is that **variables can be changed** during the execution of the program, while **constants cannot**

3.3. Integers

- ⇒ An integer is a data type that represents whole numbers (positive, negative, or zero) without any fractional or decimal parts.

```
// Algorithm 1

int main() {

int number_1= 42; // declare an integer variable named x and initialize it with the value 42

printf("The value of my number is %d\n", number_1); // print the value of x to the console

return 0; }
```

3.4. Floats / double

⇒ the **float** data type is used to represent **floating-point numbers**, which are numbers with both integer and fractional parts

```
#include <stdio.h>

int main() {
float number_2 = 3.14; // Declare a float variable named number_2 and assign it the value 3.14
printf("The value of f is %f\n", number_2); // Print the value of number_2 to the console

return 0;

}
```

3.5. Characters

⇒ characters (**chars**) are used to represent individual **characters**, such as letters, digits, symbols, or special characters

```
#include <stdio.h>

int main() {

char c = 'A'; // Declare a char variable named c and assign it the character 'A'

printf("The value of c is %c\n", c); // Print the value of c to the console

return 0;

}
```

3.6. chain of characters (String)

⇒ A sequence of characters, typically referred to as a "**string**," is a fundamental data type in programming that represents a **collection of characters**.

```
#include <stdio.h>

int main() {

char str[] = "Hello, world!"; // Declare a character array and initialize it with the string "Hello,
world!"

    printf("The string is: %s\n", str); // Print the string to the console

    return 0;

}
```

3.7. Comments

- ⇒ comments are used to add explanatory notes or remarks within the source code.
- ⇒ Comments are ignored by the compiler and are not executed as part of the program.
- ⇒ They serve as documentation for developers, making the code more readable and understandable

3.7.1. Single-line comments

```
// This is a single-line comment

int x = 42; // Assigning a value to variable x
```

3.7.2. Multi-line comments

```
/*

This is a multi-line comment.

It can extend over several lines.

*/
```

4. Conditions

⇒ Conditions, also known as **conditional statements** or **control structures**

⇒ are used in programming to **make decisions based on certain conditions**

4.1. If statement

⇒ The if statement is used to execute a block of code **if a specified condition is true**

⇒ It has the following structure:

```
if (condition) {  
    // code to execute if the condition is true  
}
```

Here's an example

```
int x = 5;  
  
if (x > 0) {  
    printf("x is positive\n"); // this message is displayed only when the value of x is greater than 0  
}
```

4.2. If-else statement

⇒ The if-else statement extends the if statement to provide an alternative block of code to execute when the condition is false.

⇒ It has the following structure:

```
if (condition) {  
    // code to execute if the condition is true  
}  
else {  
    // code to execute if the condition is false  
}
```

Here's an example

```
int x = -2; // Declare and initialize the variable x with the value -2  
  
if (x > 0) {  
    printf("x is positive\n"); // Print a message if x is greater than 0  
}  
else {  
    printf("x is non-positive\n"); // Print a message if x is not greater than 0  
}
```

4.3. Switch statement

⇒ The switch statement is used to perform different actions based on **different possible values** of a variable.

⇒ It has the following structure:

```
switch (expression) {  
    case value1:  
        // code to execute if expression matches value1  
        break;  
    case value2:
```

```
// code to execute if expression matches value2

break;

// more cases

default:

    // code to execute if expression does not match any case

}
```

⇒ Here's an example :

```
int day = 2; // Declare and initialize the variable day with the value 2

switch (day) {

    case 1:

        printf("Monday\n"); // If day equals 1, print "Monday"

        break;

    case 2:

        printf("Tuesday\n"); // If day equals 2, print "Tuesday"

        break;

    case 3:

        printf("Wednesday\n"); // If day equals 3, print "Wednesday"

        break;

    default:

        printf("Other day\n"); // If day does not match any case, print "Other day"

}
```

5. Loops

- ⇒ Loops are used in programming to repeat a block of code multiple times, allowing for efficient and repetitive execution of tasks.
- ⇒ In most programming languages, there are mainly three types of loops: **for** loop, **while** loop, and **do-while** loop.

5.1. For loop

- ⇒ The for loop is used to **repeatedly** execute a **block of code** a **specified number of times**.
- ⇒ It has the following structure:

```
for (initialization; condition; increment/decrement) {  
    // code to be executed in each iteration  
}
```

Here's an example

```
for (int i = 1; i <= 5; i++) { // the value of i start from 1 to 5 and the step is 1  
    printf("Hello world"); // Print « Hello world » 5 times  
}
```

Here's another example

```
for (int i = 1; i <= 5; i++) { // the value of i start from 1 to 5 and the step is 1 (i = 1,2,3, 4,5)  
    printf("Iteration %d\n", i); // Print the current value of i in each iteration (1,2,3, 4,5)  
}
```

- ⇒ In this example, the loop will execute five times.
- ⇒ It initializes the variable « **i** » with **1**, and as long as « **i** » is less than or equal 5 ; it executes the code block and increments i by 1 in each iteration.

Here's another example

```
for (int i = 1; i <= 10; i=i+2) { // the value of i start from 1 to 10 and the step is 2 (i= 1,3,5,7,9)
    printf("Iteration %d\n", i); // Print the current value of i in each iteration (1,3,5,7,9)
}
```

5.2. While loop

- ⇒ The while loop is used to **repeatedly** execute a block of code **as long as a condition is true**.
- ⇒ It has the following structure:

```
while (condition) {
    // code to be executed in each iteration
}
```

Here's an example

```
int i = 0; // declare a variable « i » and initialized with « 0 »
while (i < 5) {
    printf("Iteration %d\n", i); // print the value of « i »
    i++;
}
```

- ⇒ In this example, the loop will execute five times.
- ⇒ It initializes the variable « i » with 0, and as long as « i » is less than 5 it executes the code block, prints the current value of i, and increments i by 1 in each iteration.
- ⇒ The output will be the same as the for loop example. (example N°2)

5.3. Do-while loop

- ⇒ The do-while loop is used to repeatedly execute a block of code at least once, and then continue execution as long as a condition is true.
- ⇒ It has the following structure:

```
do {  
  
    // code to be executed in each iteration  
  
} while (condition);
```

Here's an example

```
int i = 0;  
  
do {  
  
    printf("Iteration %d\n", i);  
  
    i++;  
  
} while (i < 5);
```

- ⇒ In this example, the loop will execute five times.
- ⇒ It initializes the variable i with 0, executes the code block, prints the current value of i, and increments i by 1. The loop continues as long as i is less than 5

6. Arrays

- ⇒ Arrays are data structures in programming used to store a **collection of elements** of the **same type**.
- ⇒ They provide a way to group related data items under a single name.
- ⇒ In C, arrays are declared with a **fixed size** and are **indexed** starting from **0**.

Here's an example of declaring and using an array in C:

```
#include <stdio.h>

int main() {

    // Declare an array of integers with a size of 5

    int numbers[5] = {2, 4, 6, 8, 10};

    // Access and print individual elements of the array

    printf("Element at index 0: %d\n", numbers[0]); // print 2
    printf("Element at index 2: %d\n", numbers[2]); // print 6

    // Modify an element of the array

    numbers[3] = 12;

    printf("Modified element at index 3: %d\n", numbers[3]);

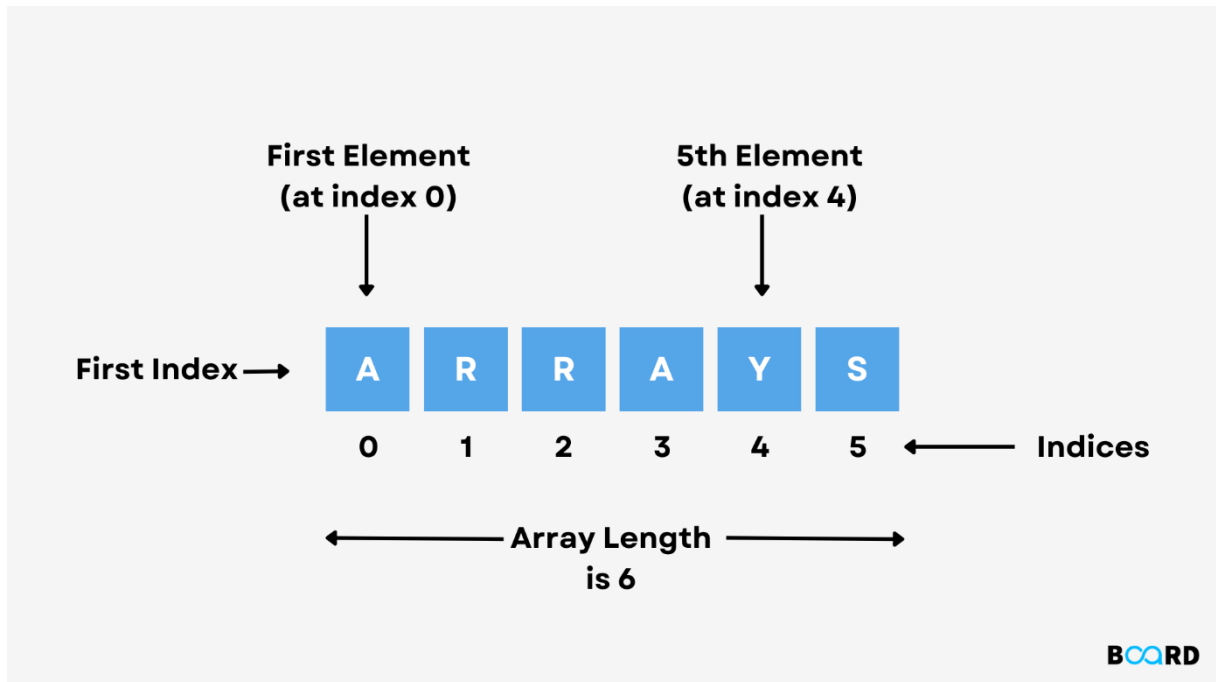
    return 0;

}
```

- ⇒ In this example, we declare an **array** named "**numbers**" that can **store five integers**. The array is **initialized** with the values 2, 4, 6, 8, and 10.
- ⇒ The elements of the array are accessed using their indices, starting from 0.
- ⇒ We use the printf function to print the values of individual array elements.

6.2. One-dimensional (1-D) arrays

⇒ One-dimensional arrays, also known as simple arrays or vectors, are arrays that store elements in a single row or a single column.



❖ Manipulate 1-D arrays

1. Initializing an array

```
int numbers[5] = {2, 4, 6, 8, 10}; // integer array named "numbers" that can store five elements
```

2. Accessing array elements

```
int x = numbers[0]; // Access the element at index 0
```

3. Modifying array elements

numbers[2] = 12; // Modify the **value** of element at **index 2**, from « **8** » to « **12** »

4. Looping through an array

⇒ This **loops** through the array and prints each element.

⇒ It starts from index 0 and continues until index 4 (inclusive).

```
for (int i = 0; i < 5; i++) {  
    printf("%d ", numbers[i]);  
}
```

5. Finding the maximum or minimum value in an array

```
int max = numbers[0];  
int min = numbers[0];  
for (int i = 1; i < 5; i++) {  
    if (numbers[i] > max) {  
        max = numbers[i];  
    }  
    if (numbers[i] < min) {  
        min = numbers[i];  
    }  
}
```

This iterates through the array and compares each element with the current maximum and minimum values, updating them if a larger maximum or smaller minimum value is found.

6.3. Two-dimensional (2-D) arrays

- ⇒ Two-dimensional arrays, also known as **matrices** or **grids**
- ⇒ Are arrays that **store elements** in a tabular form with **rows** and **columns** with the same type (int, float, char,)

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

the following figure illustrates the difference between a **one-dimensional array** and a **two-dimensional array**:

1 D ARRAY:

C	O	D	I	N	G	E	E	K
0	1	2	3	4	5	6	7	8

← single row of elements

2 D ARRAY:

		col 0	col 1	col 2
	i \ j	0	1	2
row 0	0	A	A	A
row 1	1	B	B	B
row 2	2	C	C	C

← column

} array elements

↑ rows

❖ Manipulate 2-D arrays

1. Initializing 2-D array

```
int matrix[3][4] = { {2, 4, 6, 8},{1, 3, 5, 7},{10, 20, 30, 40} };
```

⇒ This initializes a 2D array named "**matrix**" with the given values

	Column 0	Column 1	Column 2	Column 3
Row 0	2	4	6	8
Row 1	1	3	5	7
Row 2	10	20	30	40

2. Accessing array element

```
int element = matrix[1][2]; // Access the element at row 1, column 2 ( its value 5)
```

3. Modifying array element

```
matrix[2][3] = 50; // Modify the element at row 2, column 3
```

⇒ Then the new matrice values

	Column 0	Column 1	Column 2	Column 3
Row 0	2	4	6	8
Row 1	1	3	5	7
Row 2	10	20	30	50

4. Looping through array 2-D

```
for (int i = 0; i < 3; i++) { // loop to traverse all rows , row by row

    for (int j = 0; j < 4; j++) {

        printf("%d ", matrix[i][j]); // loop to traverse all columns ,one by one

    }

    printf("\n");

}
```

⇒ This **nested loop** iterates through the 2D array and prints each element.

⇒ It traverses each row and column of the array.

5. Finding the **sum** of all elements in a 2D array

```
int sum = 0; // Variable to store the sum of elements

for (int i = 0; i < 3; i++) {

    for (int j = 0; j < 4; j++) {

        sum += matrix[i][j]; // Add the element at current row and column to the sum

        // sum += matrix[i][j] is equivalent to sum =sum + matrix[i][j];

    }

}
```

Exercises

Exercise 1: Find the maximum of two integers

Exercise 2: Find the minimum of two integers

Exercise 3: Check if a number is even or odd

Exercise 4: Calculate the sum of digits in a number

Exercise 5: Calculate the average of three floating-point numbers

Exercise 6: Check if a character is a vowel or consonant

Exercise 7: Concatenate two strings

Exercise 8: Check if a string is a palindrome

Exercise 9: Find the factorial of a number

Exercise 10: Calculate the power of a number

Exercise 11: Check if a year is a leap year

Exercise 12: Check if a number is prime

Exercise 13: Print the Fibonacci sequence

Exercise 14: Swap two variables using a temporary variable

Exercise 15: Swap two variables without using a temporary variable

Exercise 16: Print the multiplication table of a number

Exercise 17: Print a pattern using nested loops

Exercise 18: Find the largest element in a 1D array

Exercise 19: Find the smallest element in a 1D array

Exercise 20: Calculate the sum of elements in a 1D array

Exercise 21: Find the transpose of a 2D array

Exercise 22: Multiply two matrices

Exercise 23: Find the sum of elements in each row of a 2D array

Exercise 24: Find the sum of elements in each column of a 2D array

Exercise 25: Check if a 2D array is symmetric

Exercise 26: Rotate a 2D array by 90 degrees

Exercise 27: Find the largest element in a 2D array

Exercise 28: Find the smallest element in a 2D array

Exercise 29: Calculate the sum of elements in a 2D array

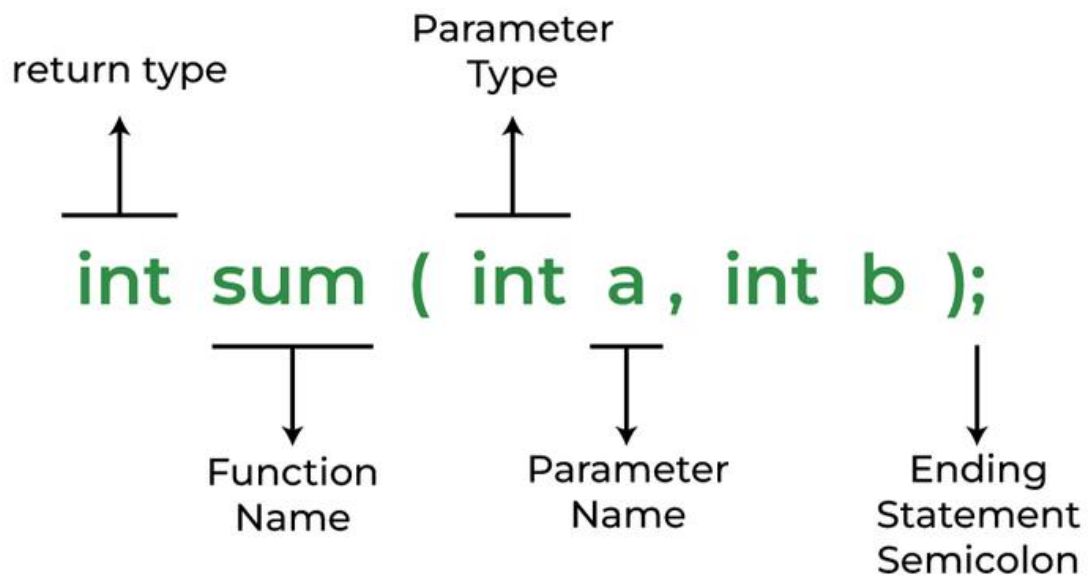
Exercise 30: Check if a 2D array is a magic square

7. Functions

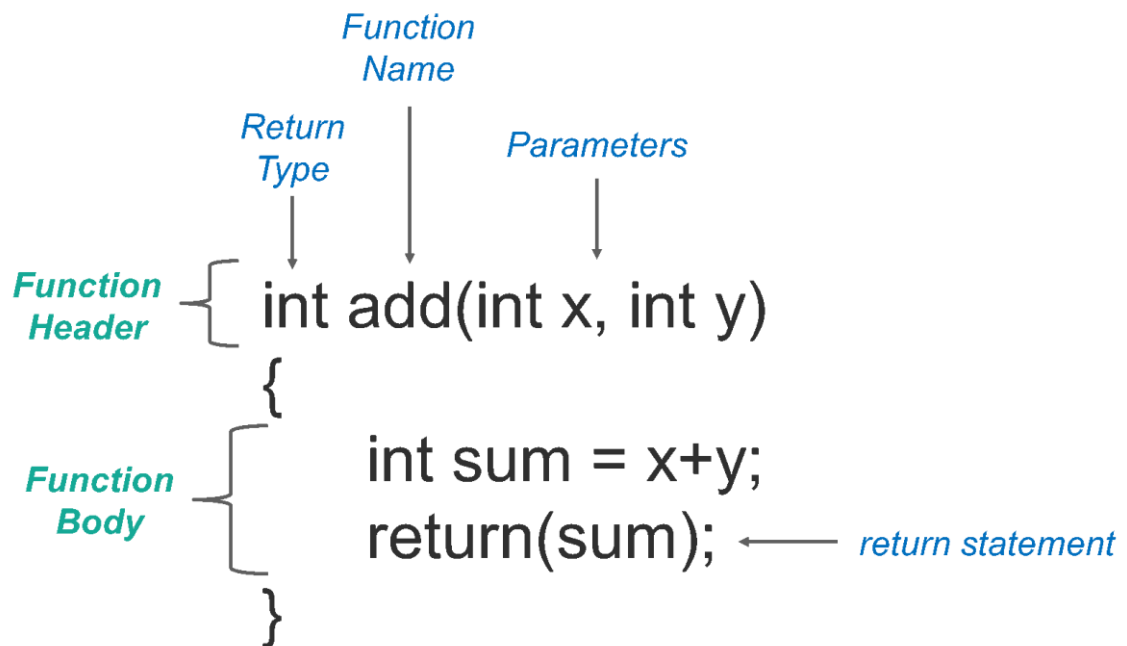
- ⇒ Functions in programming are **blocks of reusable code** that perform a specific task.
- ⇒ They allow you to **break down complex problems** into **smaller**, more manageable parts.
- ⇒ Functions **can take input parameters**, **perform operations**, and **return results**.
- ⇒ Functions are **declared with :**

1. **A return type**
2. **A name**
3. **Optionally, parameters.**

- ⇒ The general syntax for function **declaration** is as follows :



⇒ The general syntax for function **definition** is as follows :



Here's an example of a simple function that calculates the **square of a number**:

```
#include <stdio.h>

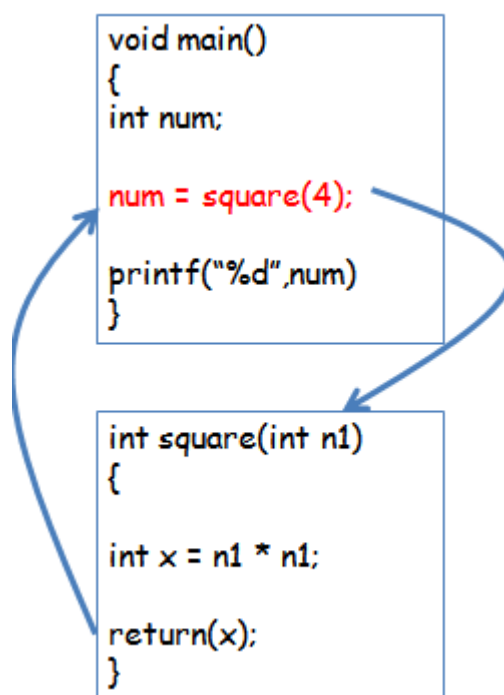
int square(int num) {
    int result = num * num;
    return result;
}

int main() {
    int x = 5;
    int square_x = square(x);
    printf("The square of %d is %d\n", x, square_x);
    return 0;
}
```

- ⇒ In this example, we **declare a function named square** that **takes an integer parameter num**.
- ⇒ Inside the function, we calculate the square of num and store it in the variable result.
- ⇒ Finally, we use the return statement to send the result back to the calling code.
- ⇒ In the main function, we define a **variable x** and assign it the value 5.
- ⇒ We then call the square function, **passing x as an argument**.
- ⇒ The **returned result is stored in the variable square_x**, and we print the output.

7.1. workflow of program when call afunction

- ⇒ When a function is called in a program, the program follows a specific workflow to execute the function and return to the point of the function call.
- ⇒ Here's a general overview of the workflow:



7.2. Example of functions

1. Function to find the maximum of two integers

```
#include <stdio.h>

int findMax(int a, int b) {

    if (a > b) { // Check if a is greater than b

        return a; // Return a if it is greater

    } else {

        return b; // Otherwise, return b

    }

}

int main() {

    int x = 5; // Define variable 'x' and assign value 5

    int y = 8; // Define variable 'y' and assign value 8

    // Call the 'findMax' function with 'x' and 'y' as arguments and store the returned value in 'max'

    int max = findMax(x, y);

    // Print the values of 'x', 'y', and 'max'

    printf("The maximum of %d and %d is %d\n", x, y, max);

    return 0;

}
```

2. Function to calculate the average of three floating-point numbers

```
#include <stdio.h>

float calculateAverage(float num1, float num2, float num3) {

    // Calculate the average of the three numbers

    float average = (num1 + num2 + num3) / 3.0;

    return average; // Return the average value

}

int main() {

    float a = 2.5; // Define variable 'a' and assign value 2.5

    float b = 3.75; // Define variable 'b' and assign value 3.75

    float c = 4.25; // Define variable 'c' and assign value 4.25

    float avg = calculateAverage(a, b, c); // Call the 'calculateAverage' function with
'a', 'b', and 'c' as arguments and store the returned value in 'avg'

    printf("The average is %.2f\n", avg); // Print the value of 'avg' with 2 decimal places

    return 0;

}
```

7.3. Exercices

1. Write a function to calculate the square of a number.
2. Write a function to calculate the cube of a number.
3. Write a function to calculate the factorial of a number.
4. Write a function to check if a number is prime.
5. Write a function to find the maximum of three numbers.
6. Write a function to find the minimum of three numbers.
7. Write a function to reverse a given string.
8. Write a function to check if a string is a palindrome.
9. Write a function to count the number of vowels in a string.
10. Write a function to count the number of words in a string.
11. Write a function to find the length of a string.
12. Write a function to convert a string to uppercase.
13. Write a function to convert a string to lowercase.
14. Write a function to calculate the sum of an integer array.
15. Write a function to find the average of an integer array.
16. Write a function to find the maximum element in an integer array.
17. Write a function to find the minimum element in an integer array.
18. Write a function to reverse an integer array.
19. Write a function to check if two arrays are equal.
20. Write a function to sort an integer array in ascending order.
21. Write a function to sort an integer array in descending order.
22. Write a function to find the transpose of a matrix.
23. Write a function to add two matrices.

24. Write a function to subtract two matrices.
25. Write a function to multiply two matrices.
26. Write a function to check if a matrix is symmetric.
27. Write a function to find the sum of the diagonal elements of a matrix.
28. Write a function to find the product of two numbers using recursion.
29. Write a function to calculate the Fibonacci sequence up to a given limit.
30. Write a function to convert a decimal number to binary.